

Convolutional Neural Networks + Neural Style Transfer

Justin Johnson

2/1/2017

Outline

- Convolutional Neural Networks
 - Convolution
 - Pooling
 - Feature Visualization
- Neural Style Transfer
 - Feature Inversion
 - Texture Synthesis
 - Style Transfer

Convolutional Neural Networks: Deep Learning with Images

IMAGENET Large Scale Visual Recognition Challenge

Steel drum

The Image Classification Challenge:

1,000 object classes

1,431,167 images



Output:

Scale

T-shirt

Steel drum

Drumstick

Mud turtle



Output:

Scale

T-shirt

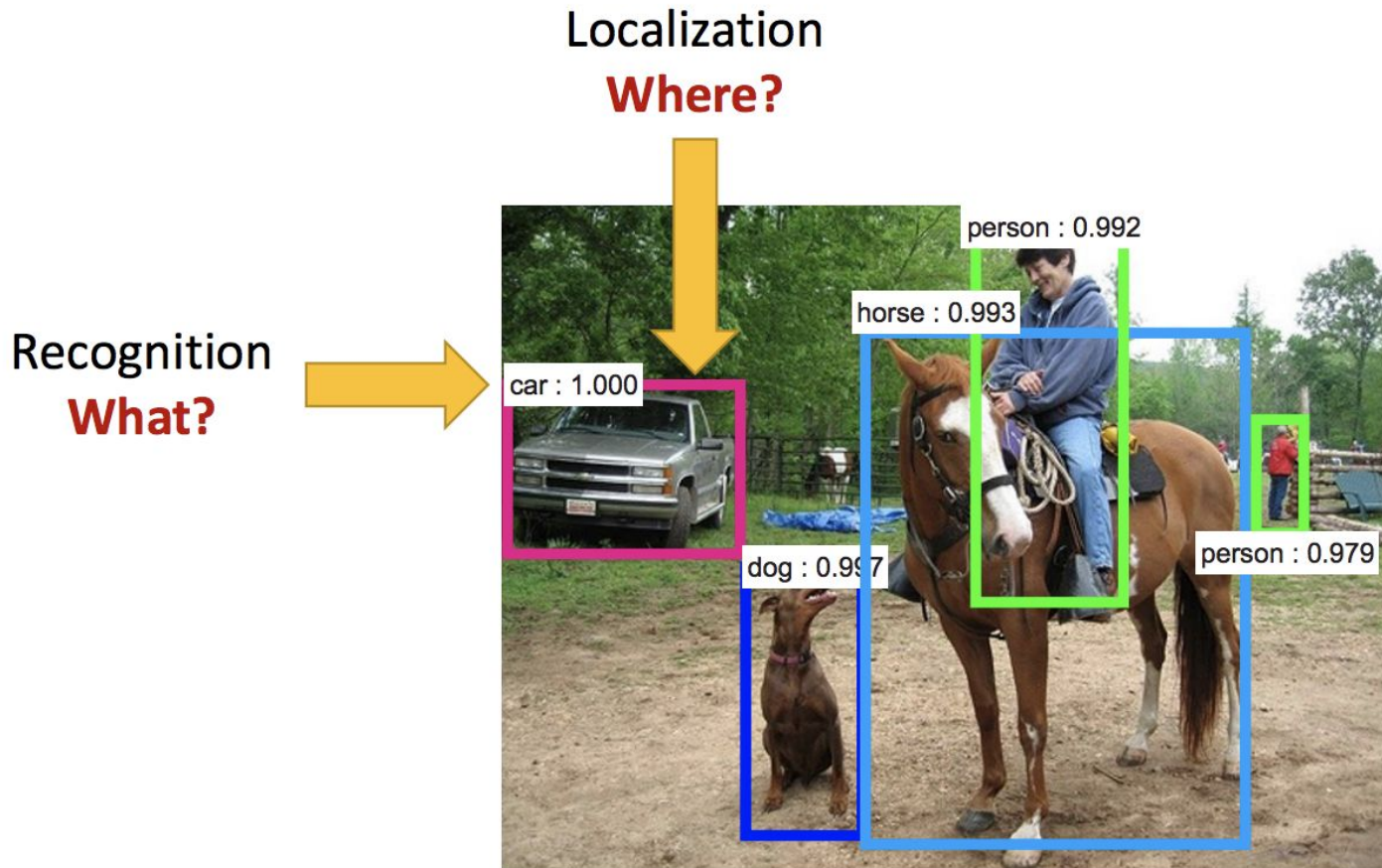
Giant panda

Drumstick

Mud turtle



Object Detection = What, and Where



Object segmentation

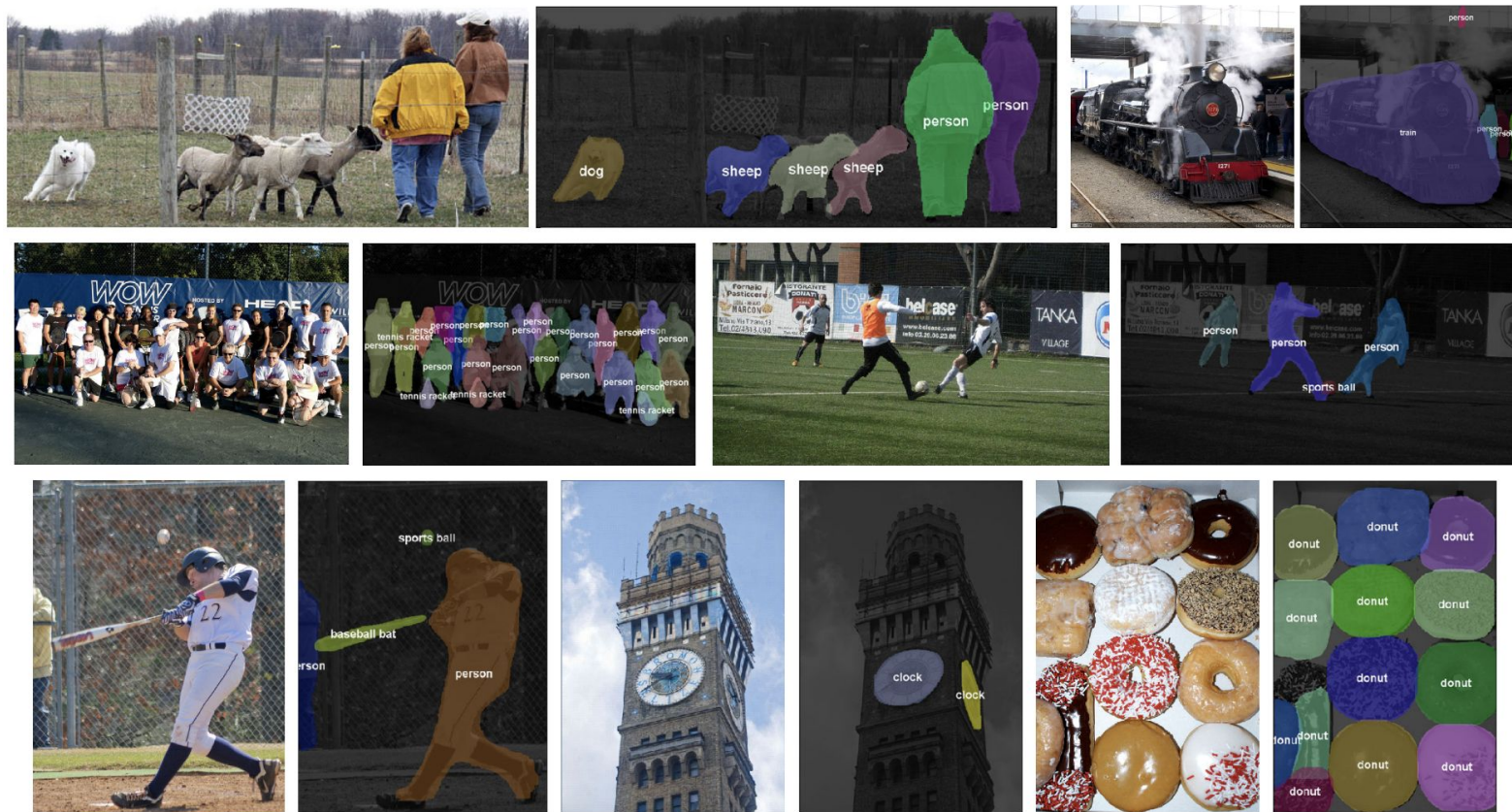


Figure credit: Dai, He, and Sun, "Instance-aware Semantic Segmentation via Multi-task Network Cascades", CVPR 2016

Pose Estimation

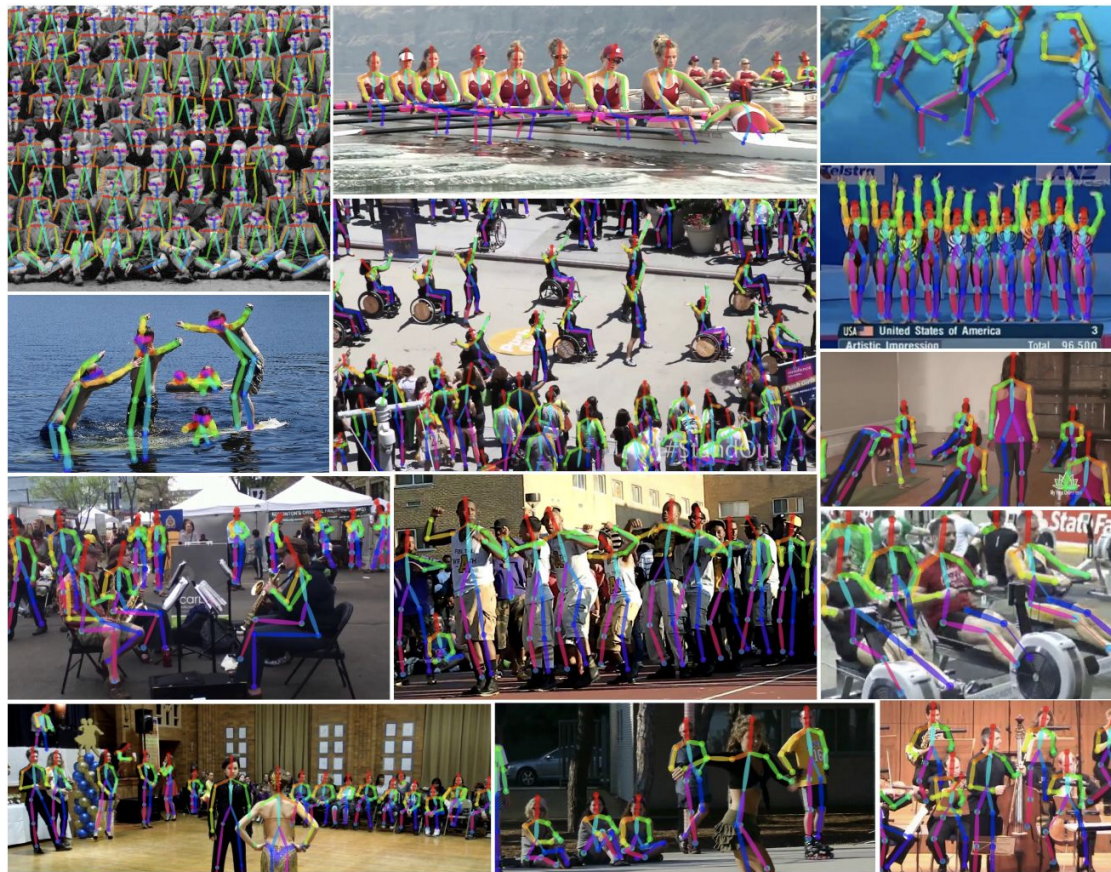
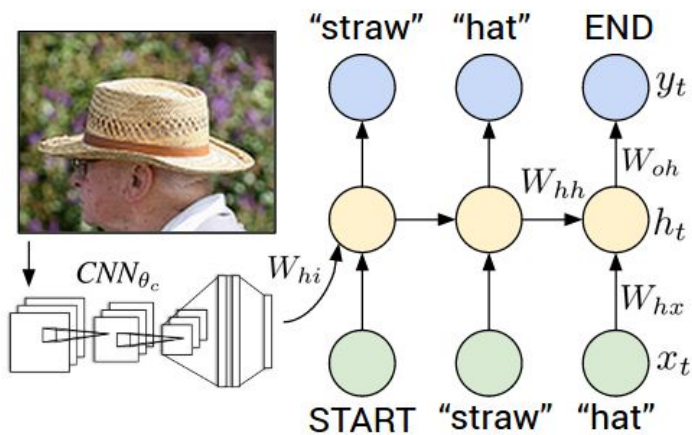
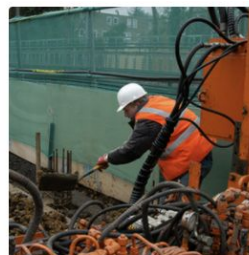


Figure credit: Cao et al, "Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields", arXiv 2016

Image Captioning



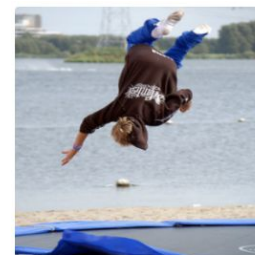
"man in black shirt is playing guitar."



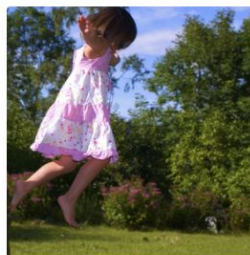
"construction worker in orange safety vest is working on road."



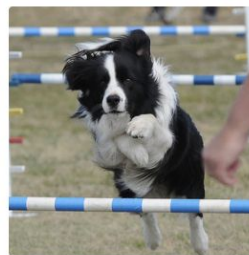
"two young girls are playing with lego toy."



"boy is doing backflip on wakeboard."



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."



"young girl in pink shirt is swinging on swing."



"man in blue wetsuit is surfing on wave."

Visual Question Answering



What color are her eyes?
What is the mustache made of?



How many slices of pizza are there?
Is this a vegetarian pizza?



Is this person expecting company?
What is just under the tree?



Does it appear to be rainy?
Does this person have 20/20 vision?

Figure credit: Agrawal et al, "VQA: Visual Question Answering", ICCV 2015

Image			
	Q: Who is behind the batter? A: Catcher. A: Umpire. A: Fans. A: Ball girl.	Q: What adorns the tops of the post? A: Gulls. A: An eagle. A: A crown. A: A pretty sign.	Q: How many cameras are in the photo? A: One. A: Two. A: Three. A: Four.
Multiple Choices	H: Catcher. ✓ M: Umpire. ✗	H: Gulls. ✓ M: Gulls. ✓	H: Three. ✗ M: One. ✓
	H: Catcher. ✓ M: Catcher. ✓	H: Gulls. ✓ M: A crown. ✗	H: One. ✓ M: One. ✓
w/ Image w/o Image			
	Q: Why is there rope? A: To tie up the boats. A: To tie up horses. A: To hang people. A: To hit tether balls.	Q: What kind of stuffed animal is shown? A: Teddy Bear. A: Monkey. A: Tiger. A: Bunny rabbit.	Q: What animal is being petted? A: A sheep. A: Goat. A: Alpaca. A: Pig.
	H: To hit tether balls. ✗ M: To hang people. ✗	H: Monkey. ✗ M: Teddy Bear. ✓	H: A sheep. ✓ M: A sheep. ✓
	H: To tie up the boats. ✓ M: To hang people. ✗	H: Teddy Bear. ✓ M: Teddy Bear. ✓	H: Goat. ✗ M: A sheep. ✓

Figure credit: Zhu et al, "Visual7W: Grounded Question Answering in Images", CVPR 2016

Image Super-Resolution

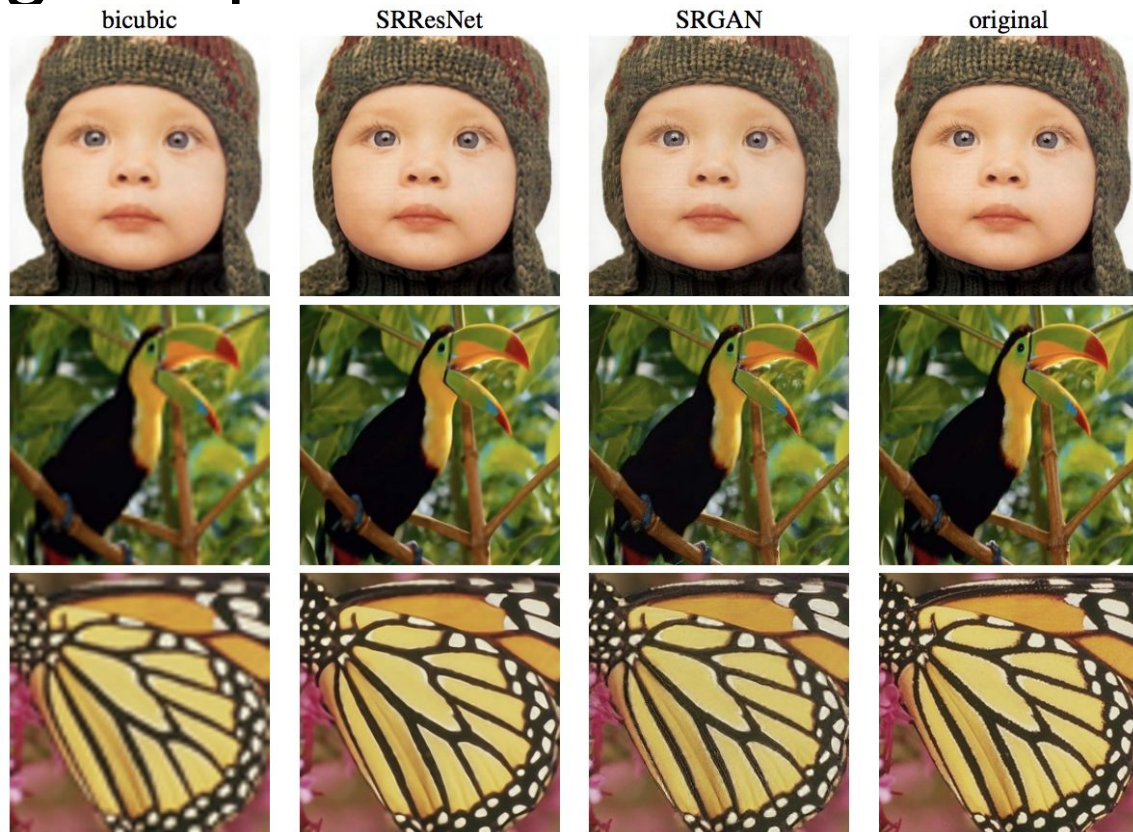


Figure credit: Ledig et al, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network", arXiv 2016

Generating Art

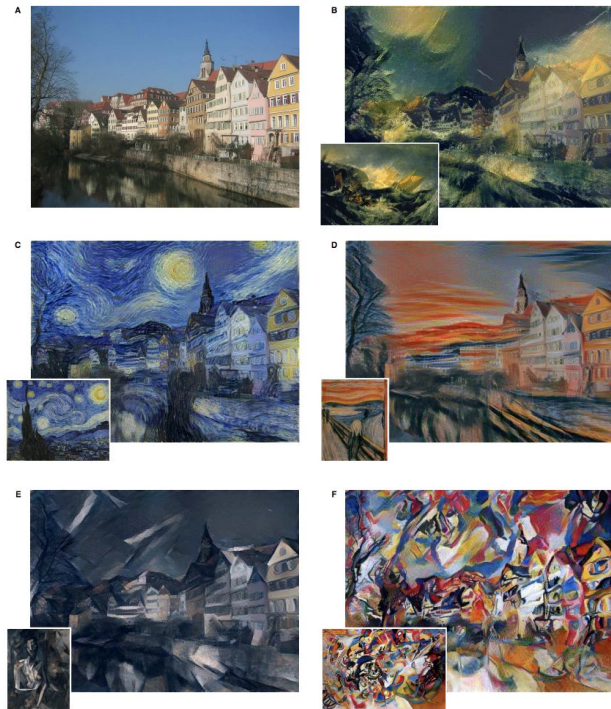


Figure credit: Gatys, Ecker, and Bethge, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016

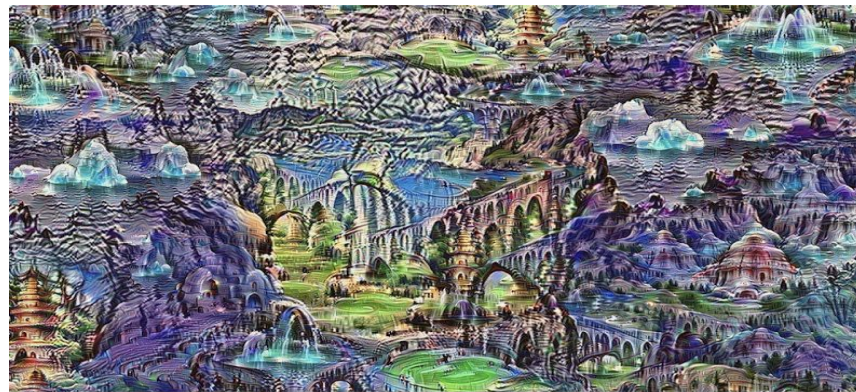


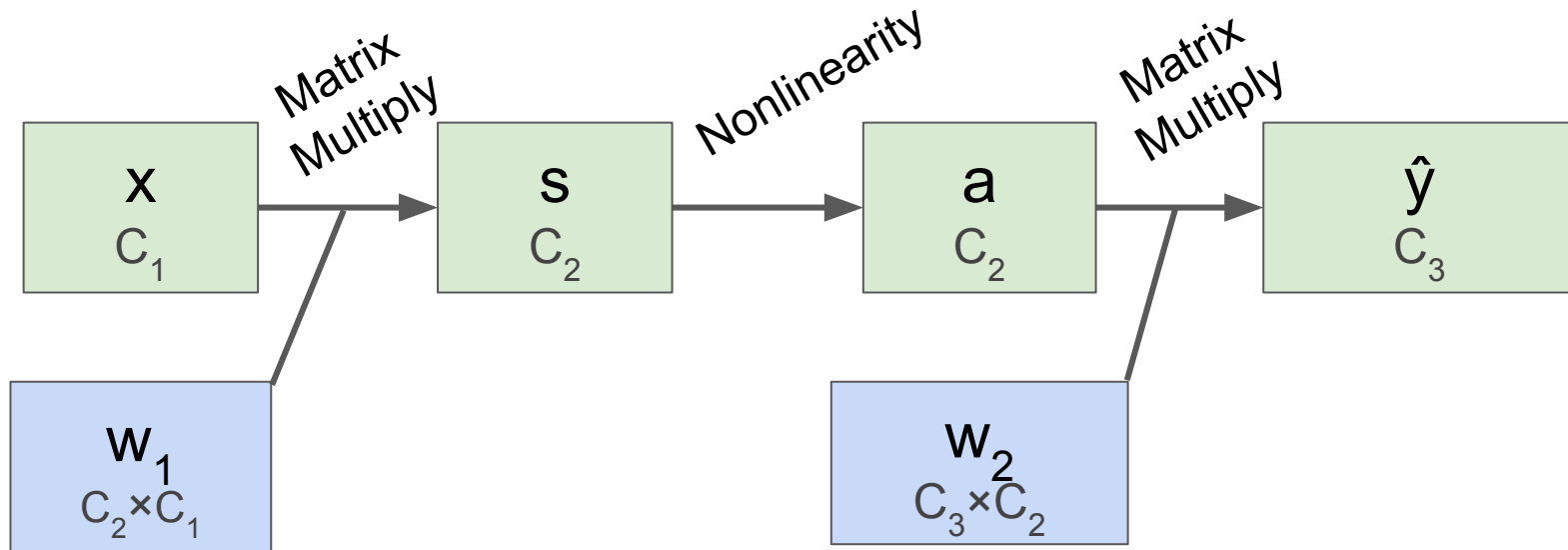
Figure credit: Mordvintsev, Olah, and Tyka, "Inceptionism: Going Deeper into Neural Networks", <https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html>



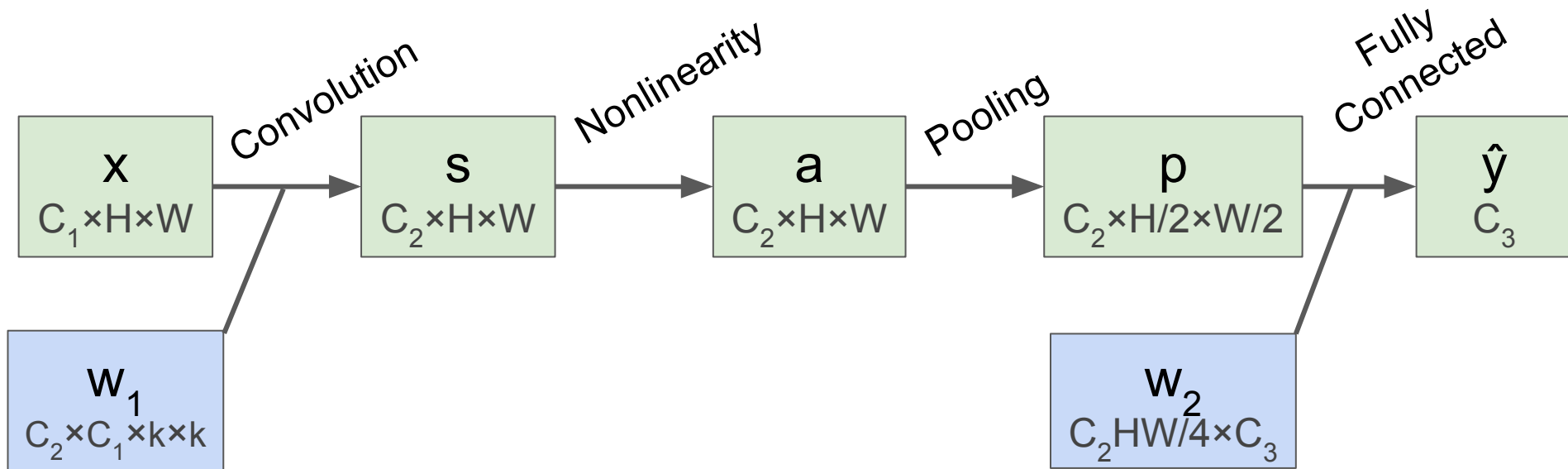
Figure credit: Johnson, Alahi, and Fei-Fei: "Perceptual Losses for Real-Time Style Transfer and Super-Resolution", ECCV 2016, <https://github.com/jcjohnson/fast-neural-style>

What is a Convolutional Neural Net?

Fully-Connected Neural Network

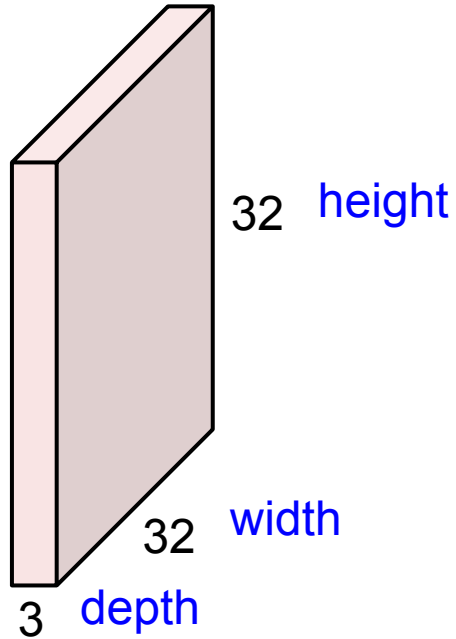


Convolutional Neural Network



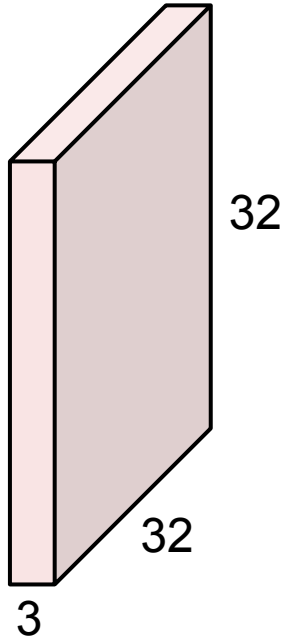
Convolution Layer

32x32x3 image



Convolution Layer

32x32x3 image



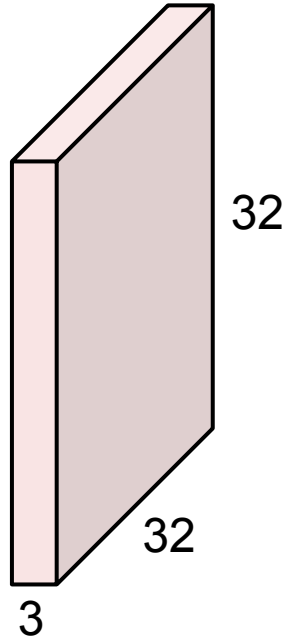
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

32x32x3 image



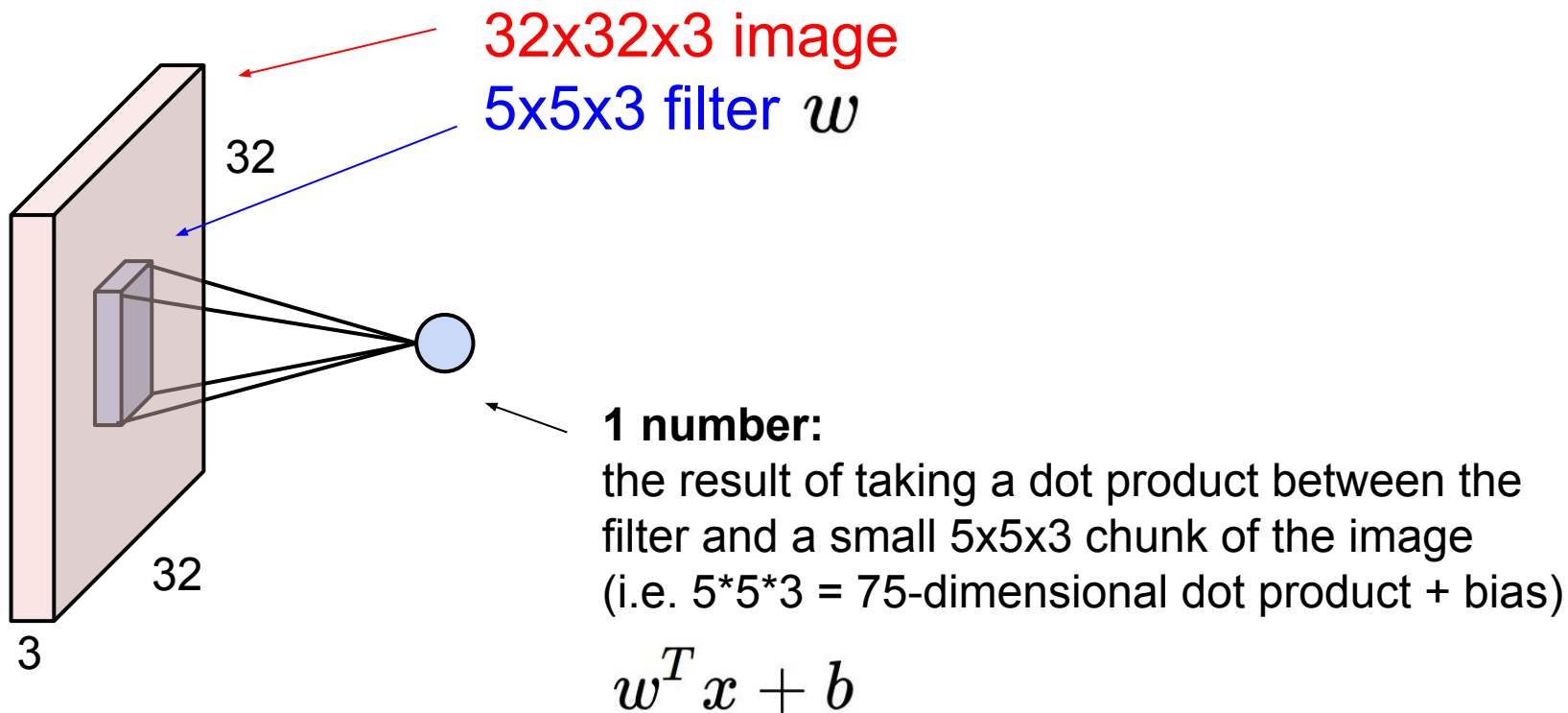
Filters always extend the full depth of the input volume

5x5x3 filter

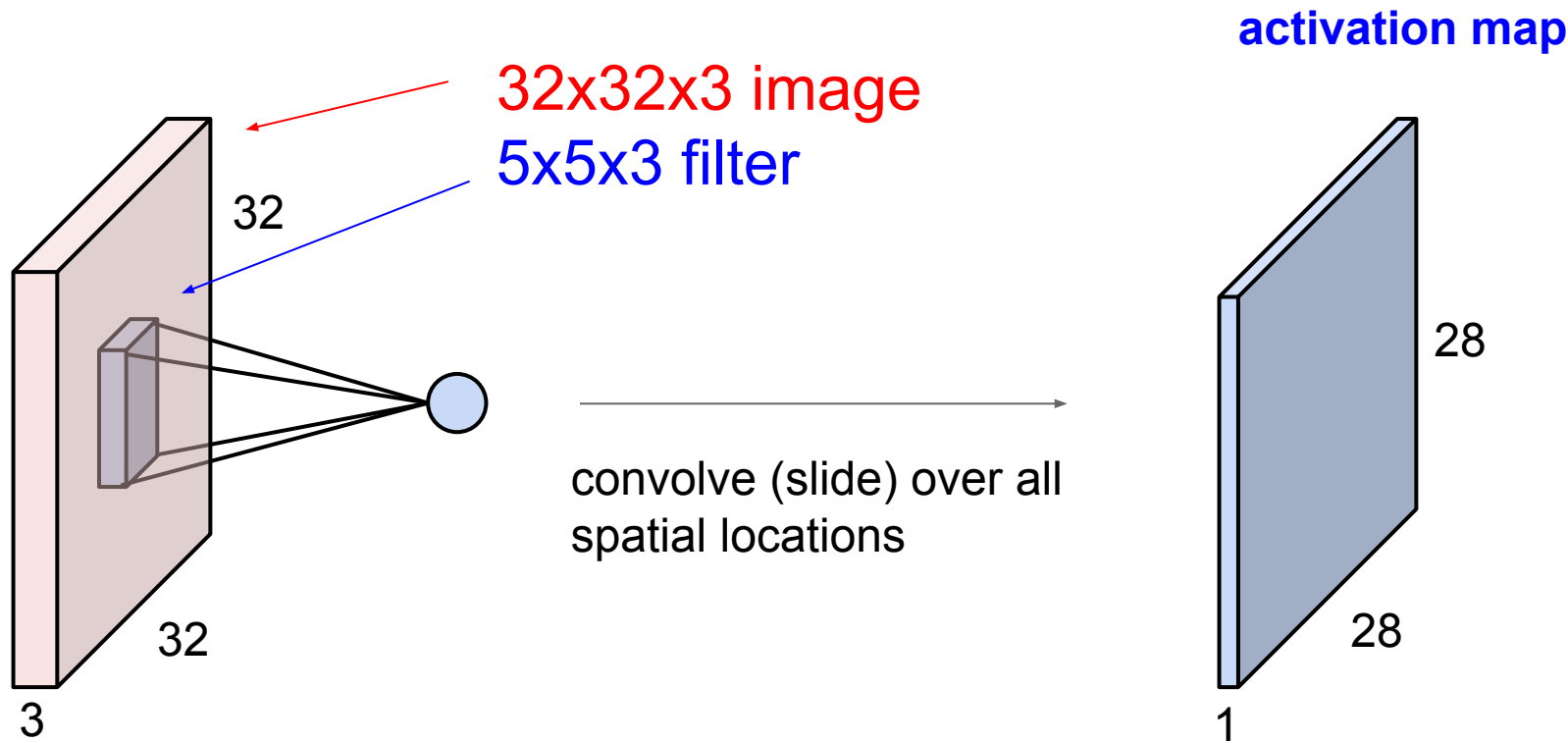


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

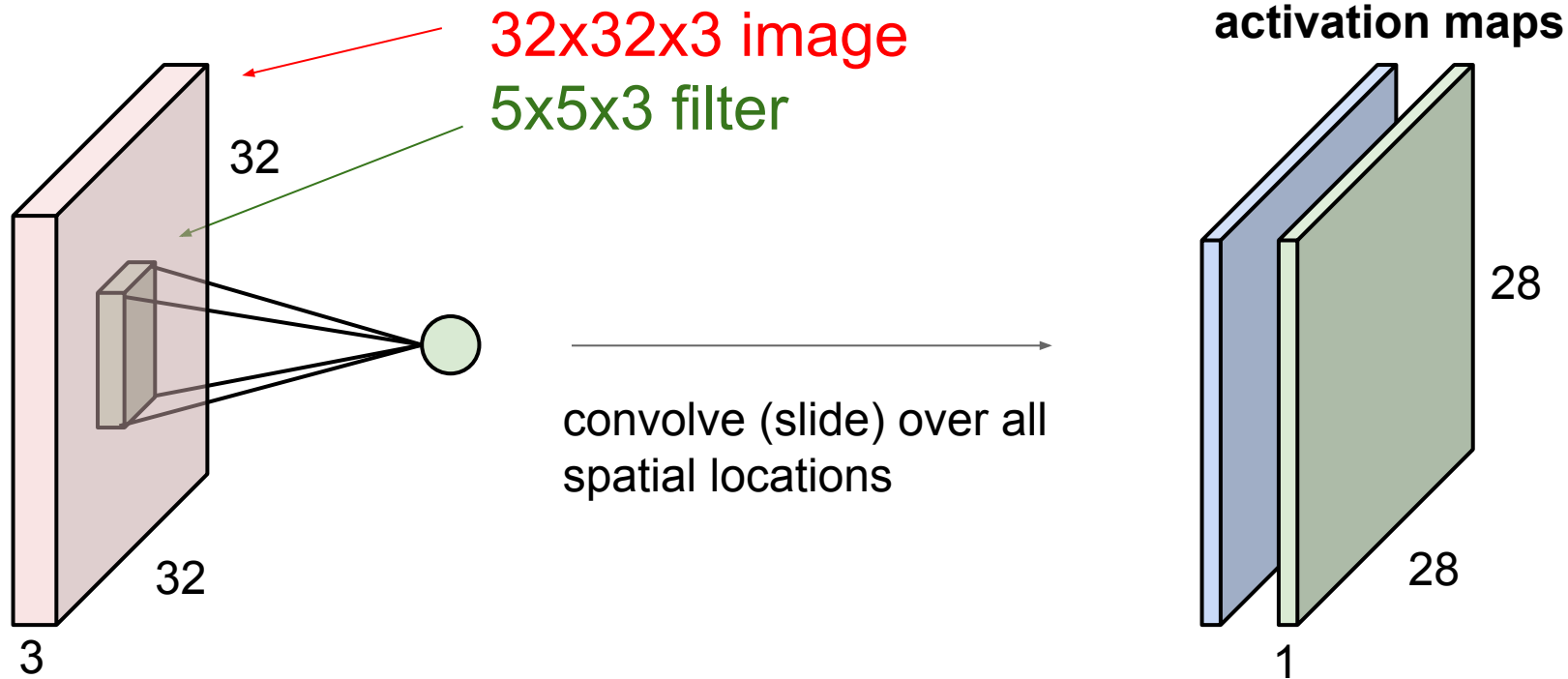


Convolution Layer

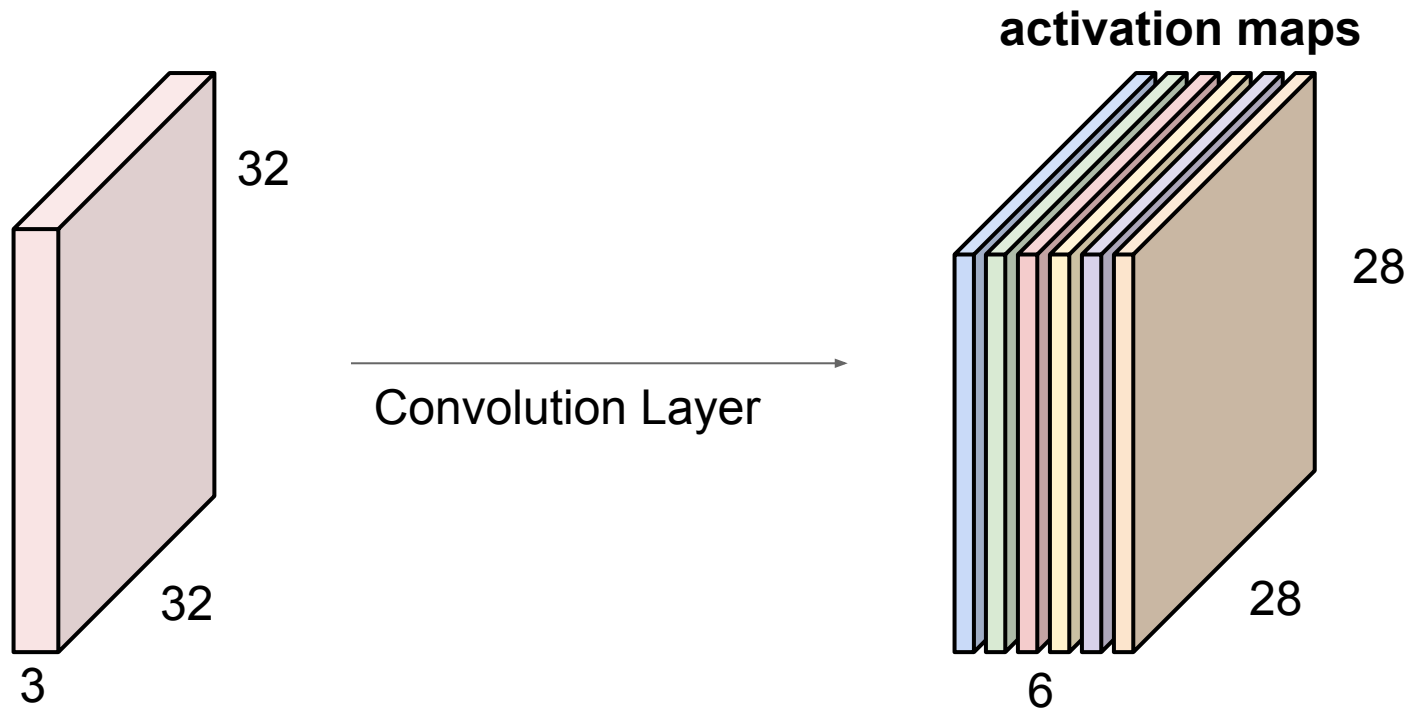


Convolution Layer

consider a second, **green** filter



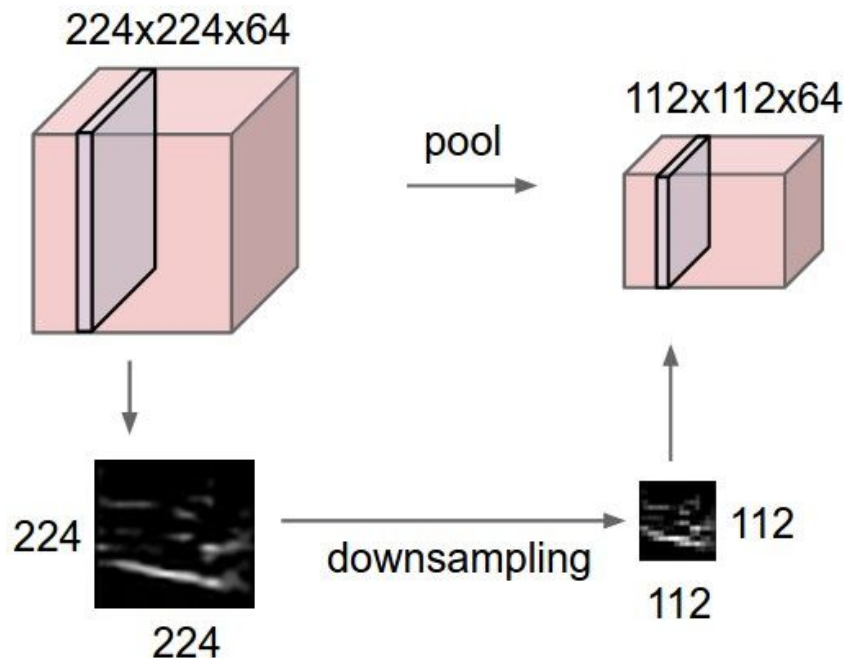
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



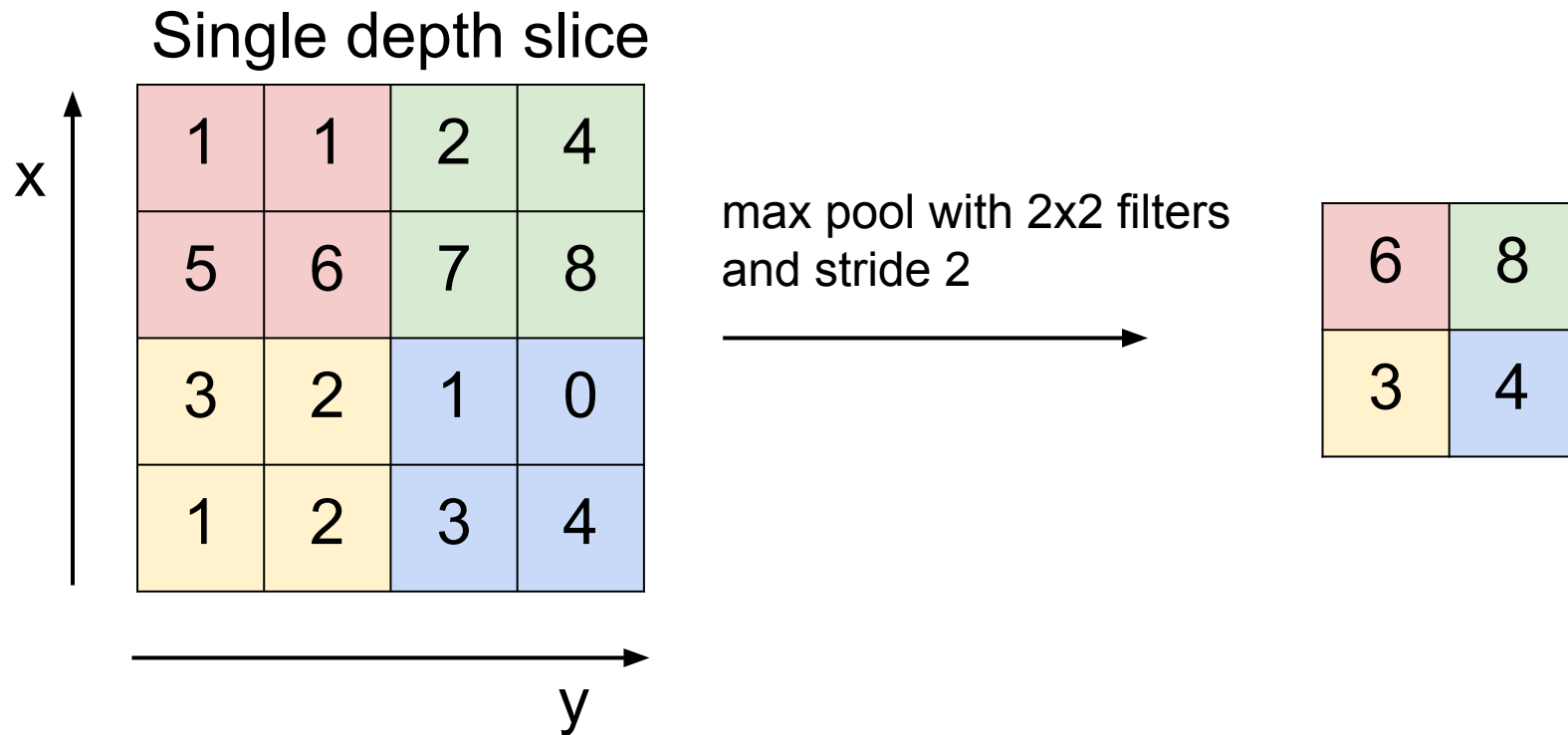
We stack these up to get a “new image” of size 28x28x6!

Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:

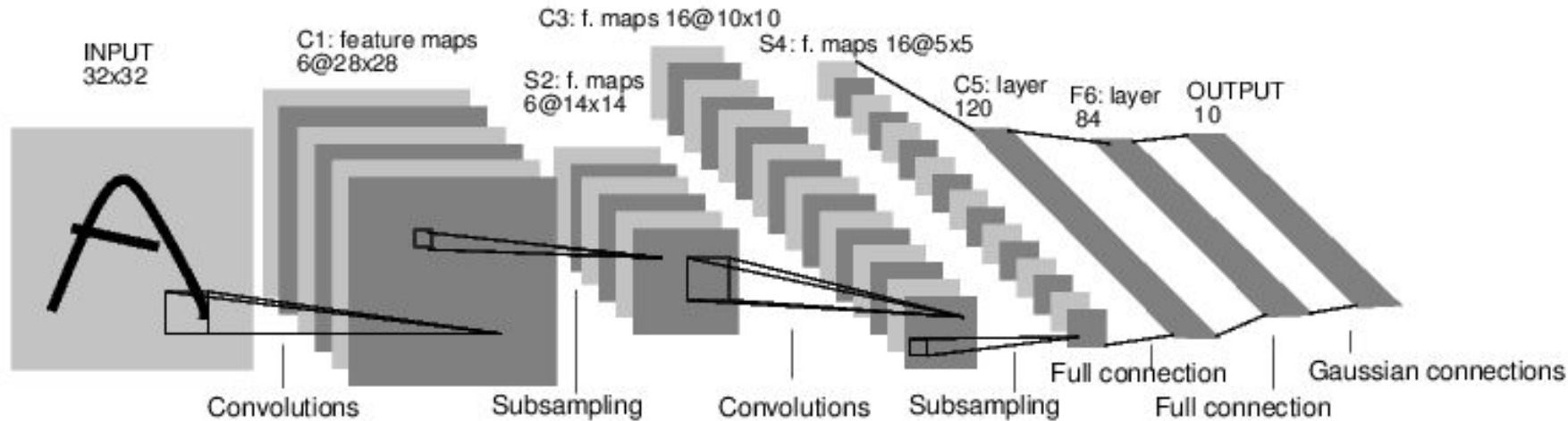


MAX POOLING



Case Study: LeNet-5

[LeCun et al., 1998]



Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

Case Study: AlexNet

[Krizhevsky et al. 2012]

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

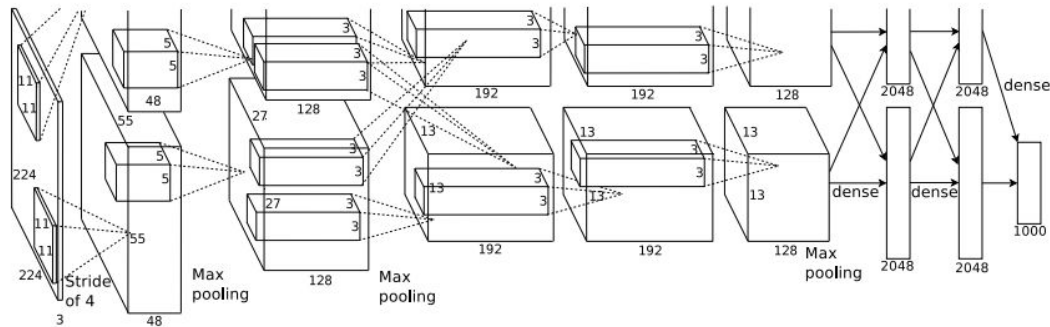
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

best model

11.2% top 5 error in ILSVRC 2013

->

7.3% top 5 error

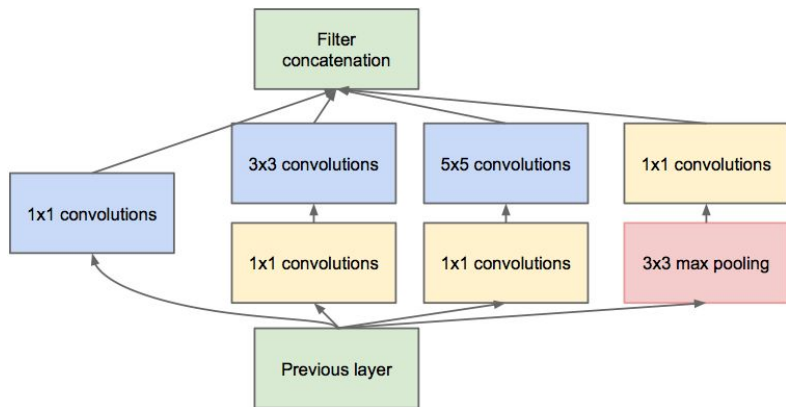
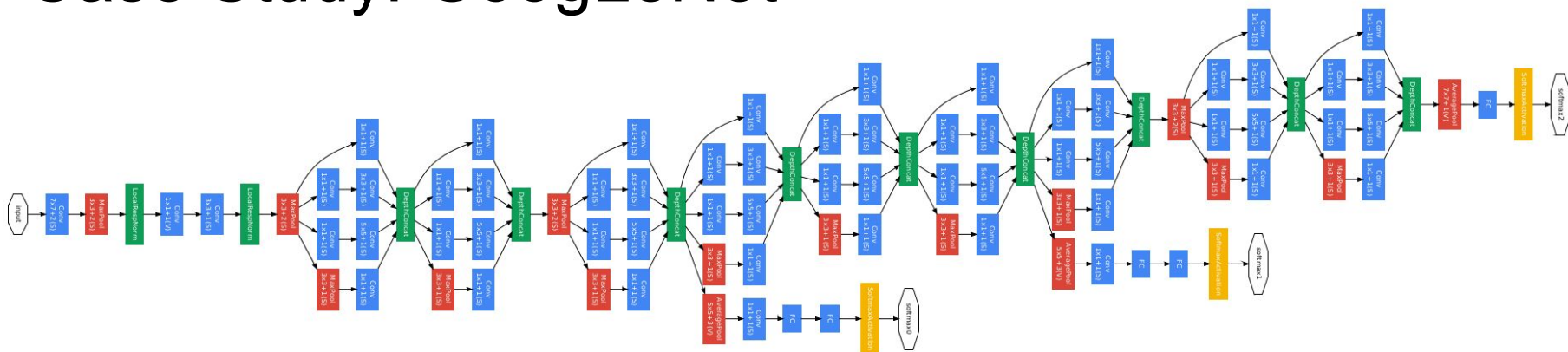
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Case Study: GoogLeNet

[Szegedy et al., 2014]

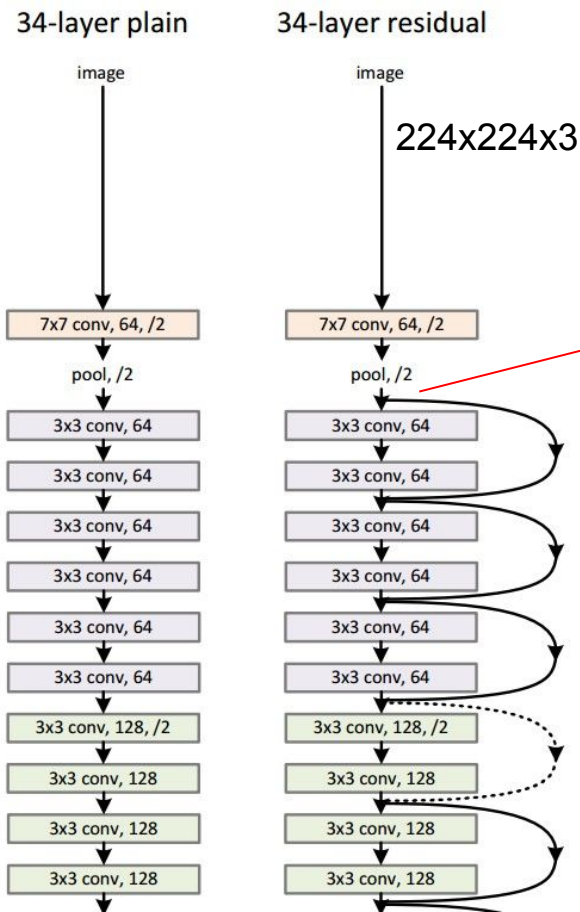


Inception module

ILSVRC 2014 winner (6.7% top 5 error)

Case Study: ResNet

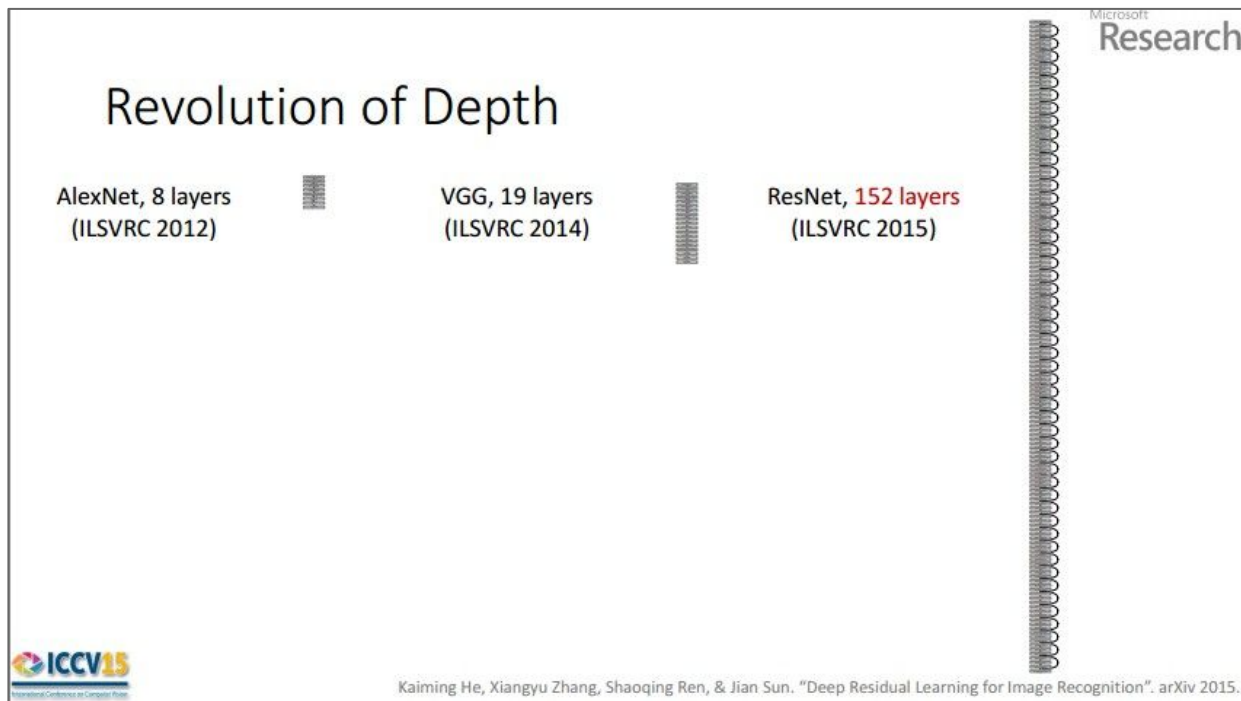
[He et al., 2015]



spatial dimension
only 56x56!

Case Study: ResNet [He et al., 2015]

ILSVRC 2015 winner (3.6% top 5 error)

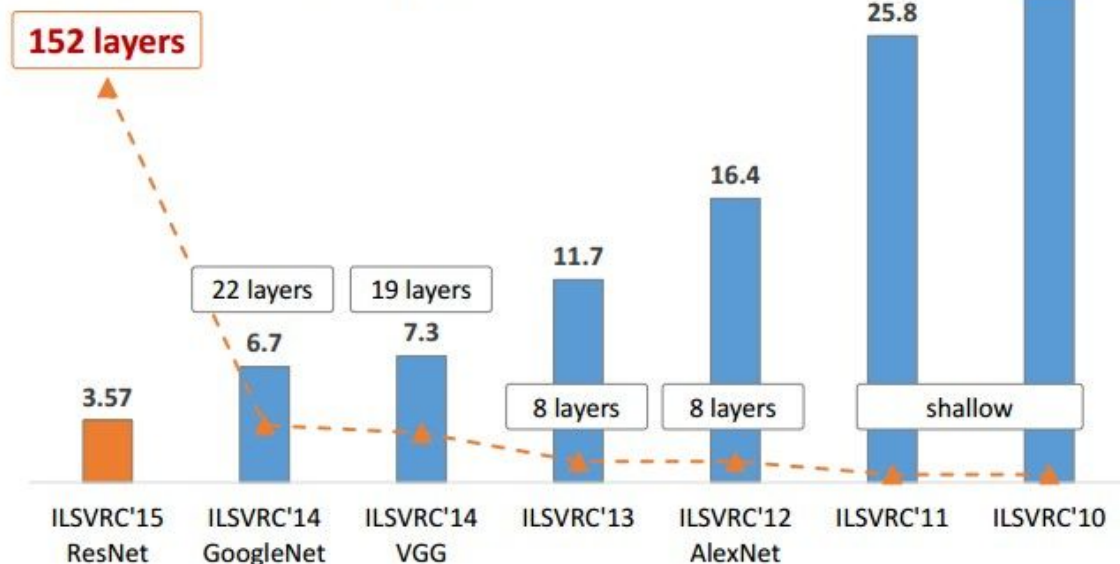


2-3 weeks of training
on 8 GPU machine

at runtime: faster
than a VGGNet!
(even though it has
8x more layers)

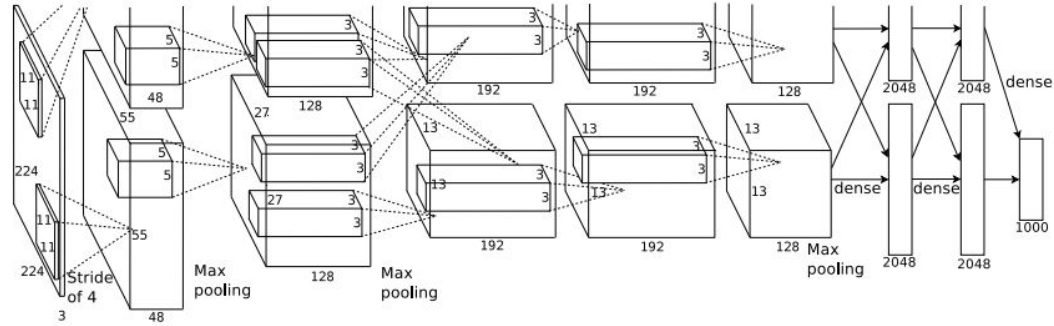
(slide from Kaiming He's ICCV 2015 presentation)

Revolution of Depth



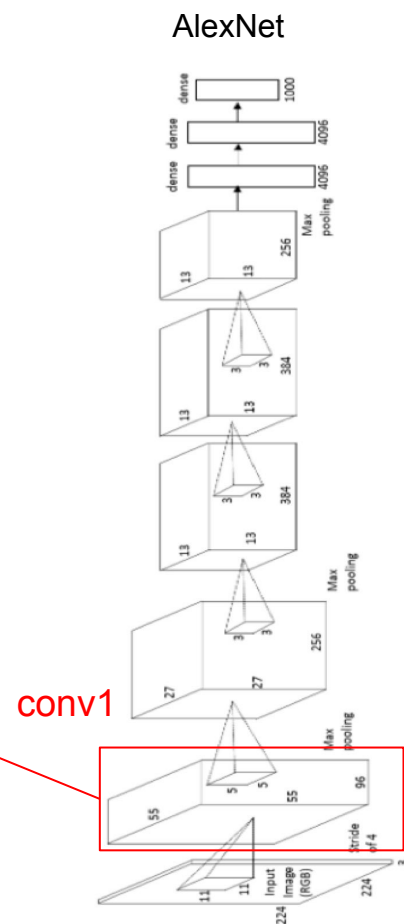
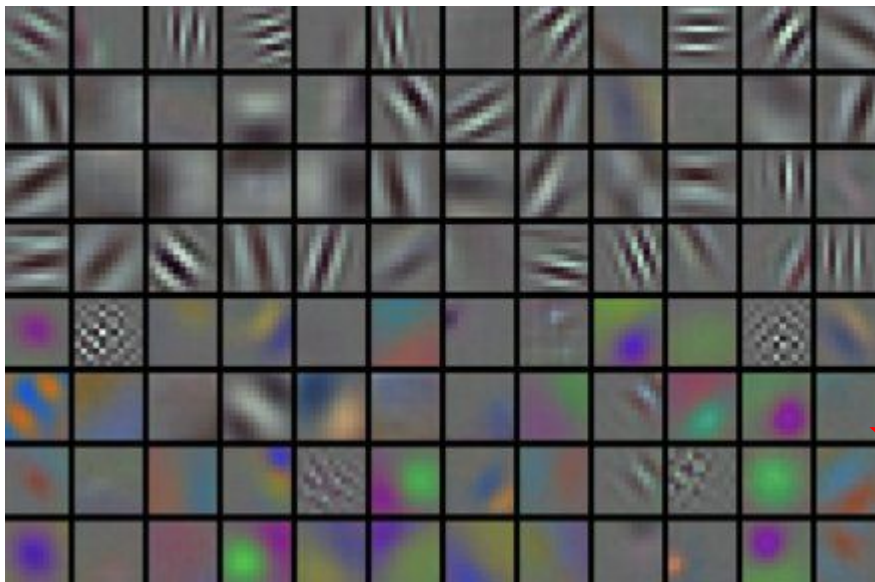
ImageNet Classification top-5 error (%)

(slide from Kaiming He's ICCV 2015 presentation)

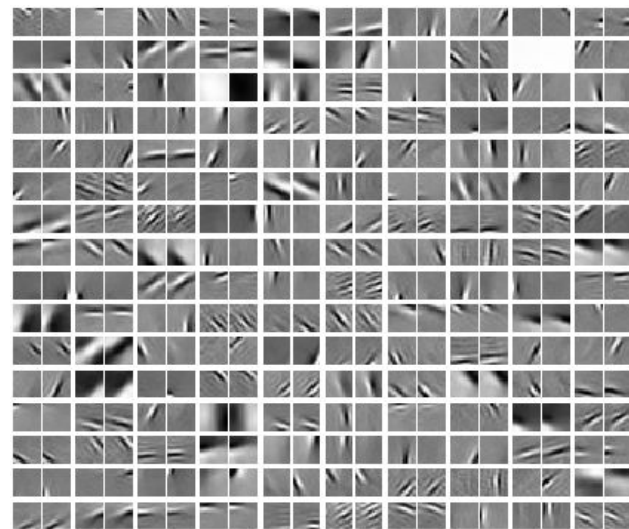
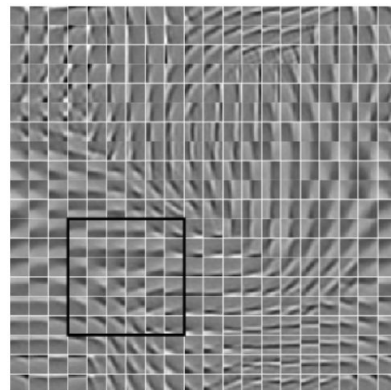
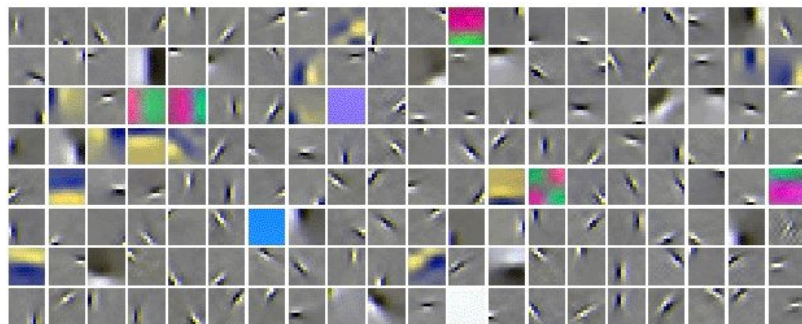
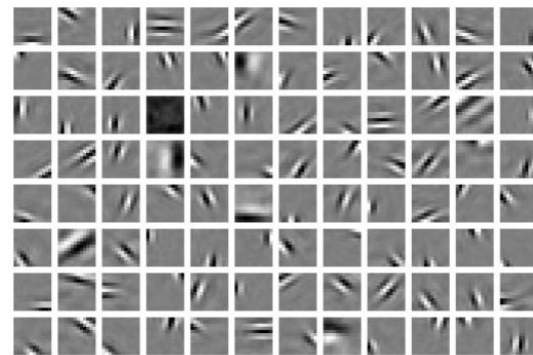
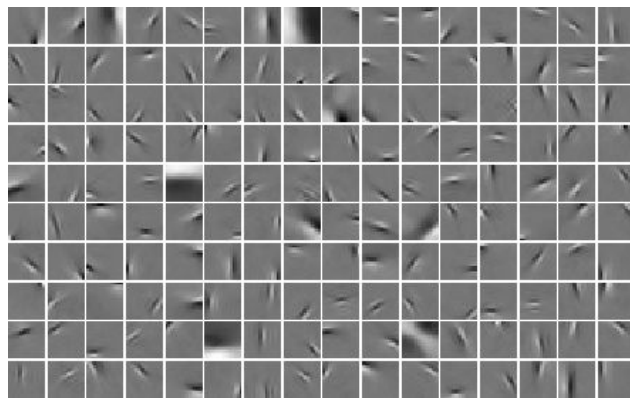
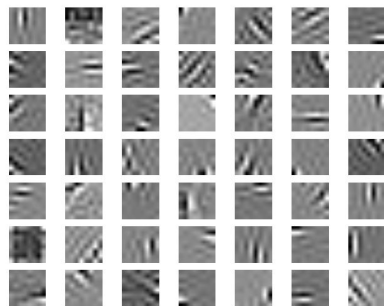


Visualizing ConvNet Features

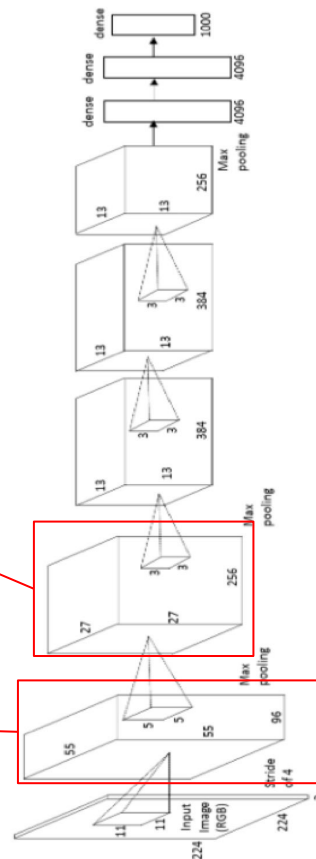
Visualizing CNN features: Look at filters



Many networks learn similar filters



Visualizing CNN features: Look at filters



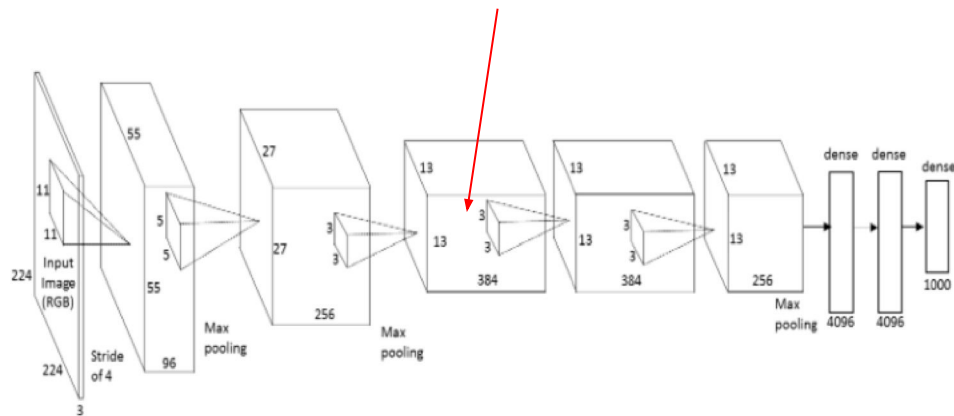
Filters from higher layers don't make much sense

Visualizing CNN features: (Guided) Backprop

Choose an image



Choose a layer and a neuron in a CNN

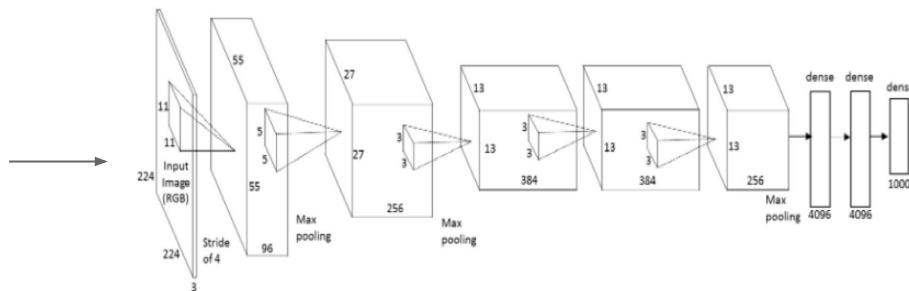


Question:

How does the chosen neuron respond to the image?

Visualizing CNN features: (Guided) Backprop

1. Feed image into net



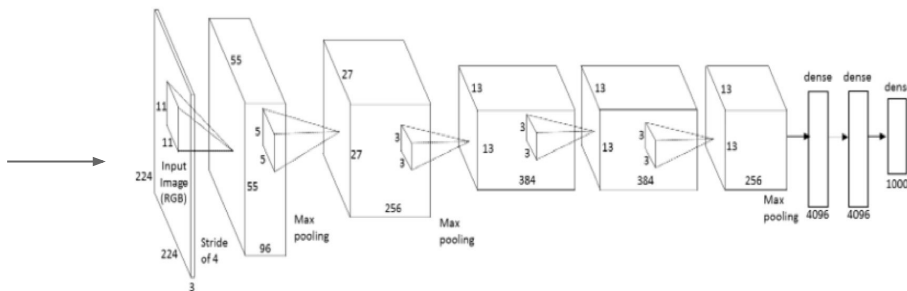
Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

Dosovitskiy et al, "Striving for Simplicity: The All Convolutional Net", ICLR Workshop 2015

Slide credit: CS231n Lecture 9

Visualizing CNN features: (Guided) Backprop

1. Feed image into net



2. Set gradient of chosen layer to all zero, except 1 for the chosen neuron

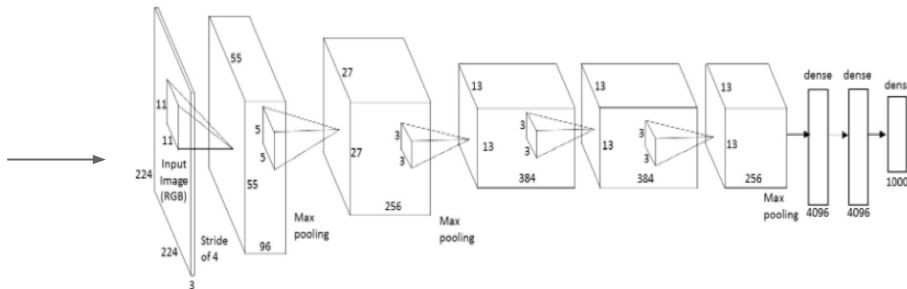
Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

Dosovitskiy et al, "Striving for Simplicity: The All Convolutional Net", ICLR Workshop 2015

Slide credit: CS231n Lecture 9

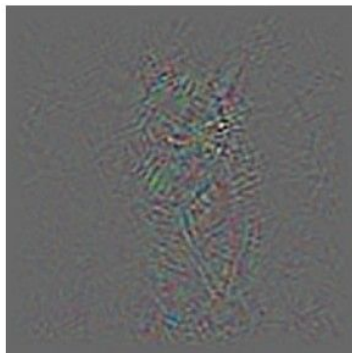
Visualizing CNN features: (Guided) Backprop

1. Feed image into net



2. Set gradient of chosen layer to all zero, except 1 for the chosen neuron

3. Backprop to image:



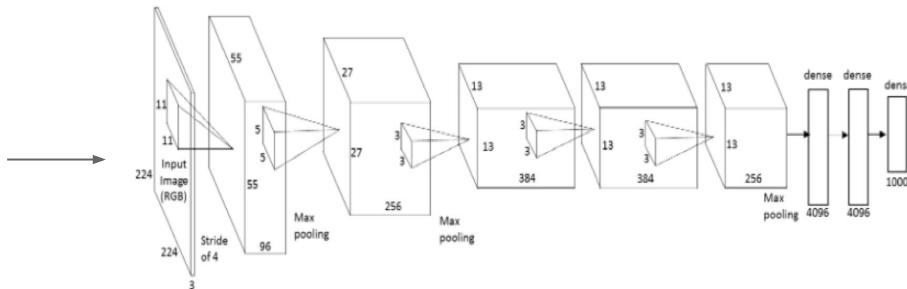
Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

Dosovitskiy et al, "Striving for Simplicity: The All Convolutional Net", ICLR Workshop 2015

Slide credit: CS231n Lecture 9

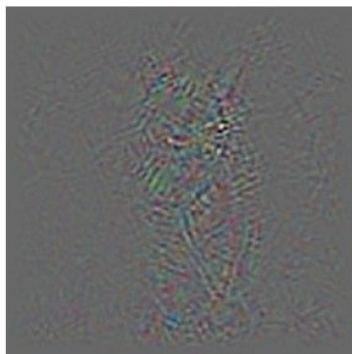
Visualizing CNN features: (Guided) Backprop

1. Feed image into net

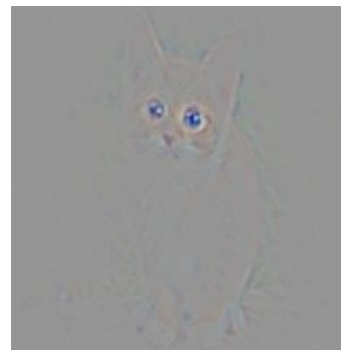


2. Set gradient of chosen layer to all zero, except 1 for the chosen neuron

3. Backprop to image:



**Guided
backpropagation:
instead**

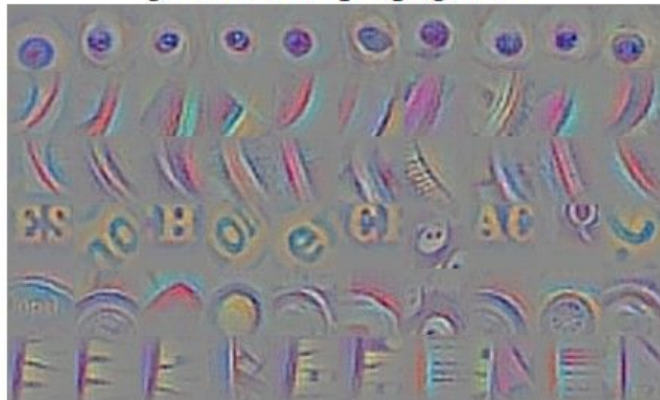


Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

Dosovitskiy et al, "Striving for Simplicity: The All Convolutional Net", ICLR Workshop 2015

Slide credit: CS231n Lecture 9

guided backpropagation



corresponding image crops



Visualization of patterns learned by the layer **conv6** (top) and layer **conv9** (bottom) of the network trained on ImageNet.

Each row corresponds to one filter.

guided backpropagation



corresponding image crops



The visualization using “guided backpropagation” is based on the top 10 image patches activating this filter taken from the ImageNet dataset.

Visualizing CNN features: Gradient Ascent

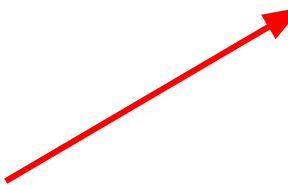
(Guided) backprop:

Find the part of an image that a neuron responds to

Gradient ascent:

Generate a synthetic image that maximally activates a neuron

$$I^* = \arg \max_I \boxed{f(I)} + \boxed{R(I)}$$



Neuron value



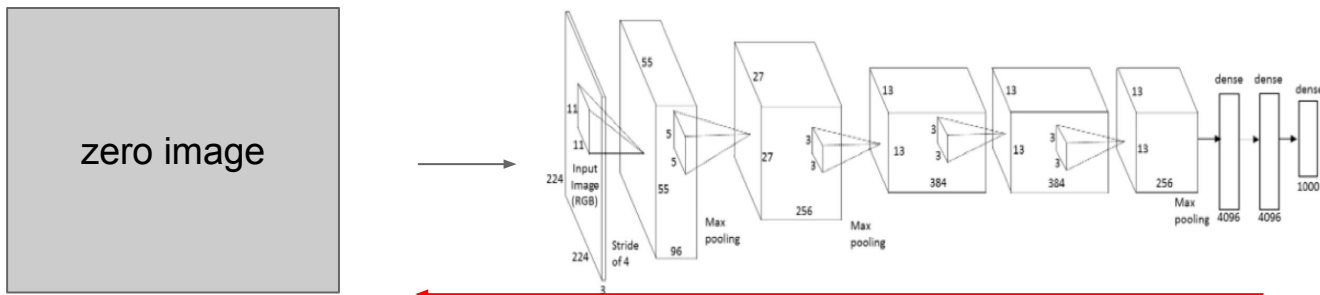
Natural image regularizer

Visualizing CNN features: Gradient Ascent

$$\arg \max_I \boxed{S_c(I)} - \lambda \|I\|_2^2$$

score for class c (before Softmax)

1. Initialize image to zeros



Repeat:

2. Forward image to compute current scores
3. Set gradient of scores to be 1 for target class, 0 for others
4. Backprop to get gradient on image
5. Make a small update to the image

Visualizing CNN features: Gradient Ascent



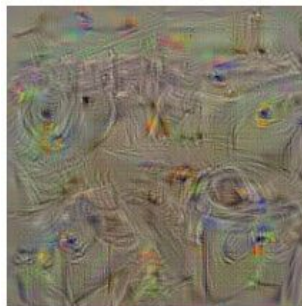
dumbbell



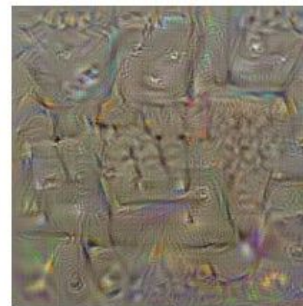
cup



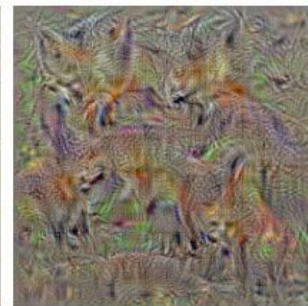
dalmatian



washing machine



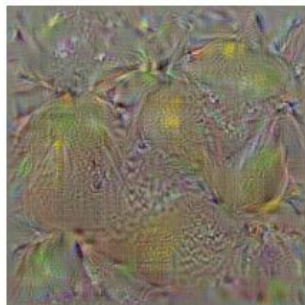
computer keyboard



kit fox



bell pepper



lemon



husky



goose



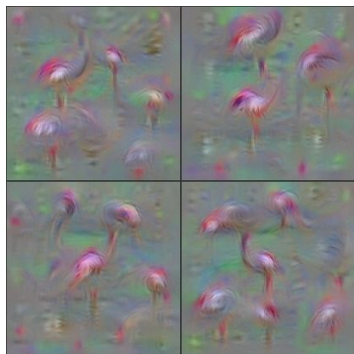
ostrich



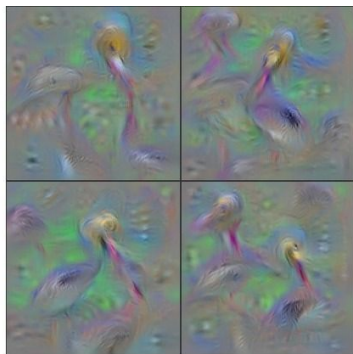
limousine

Visualizing CNN features: Gradient Ascent

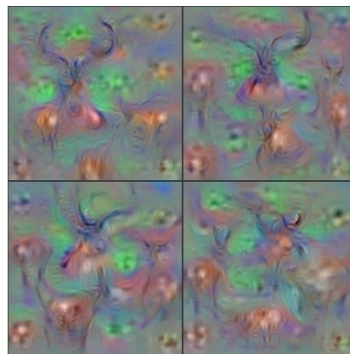
Better image regularizers give prettier results:



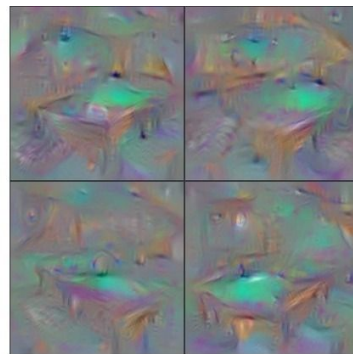
Flamingo



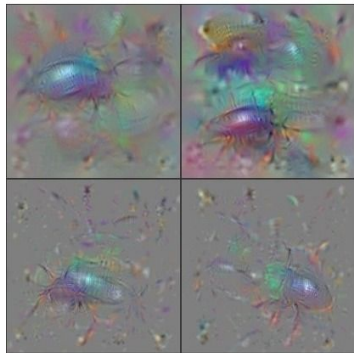
Pelican



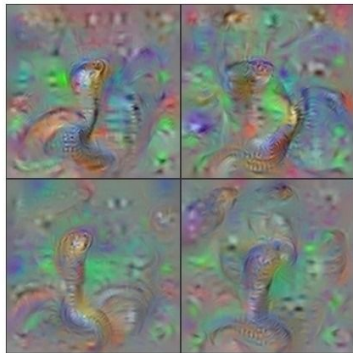
Hartbeest



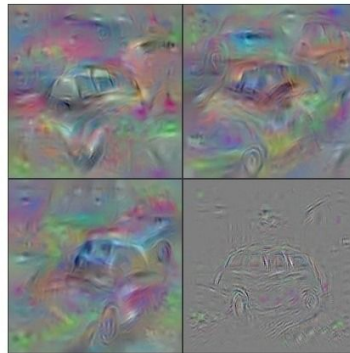
Billiard Table



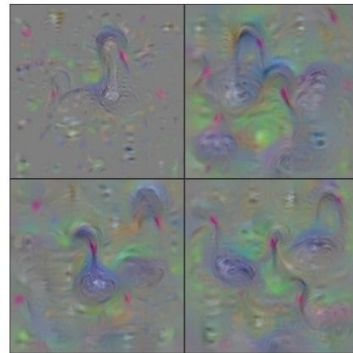
Ground Beetle



Indian Cobra



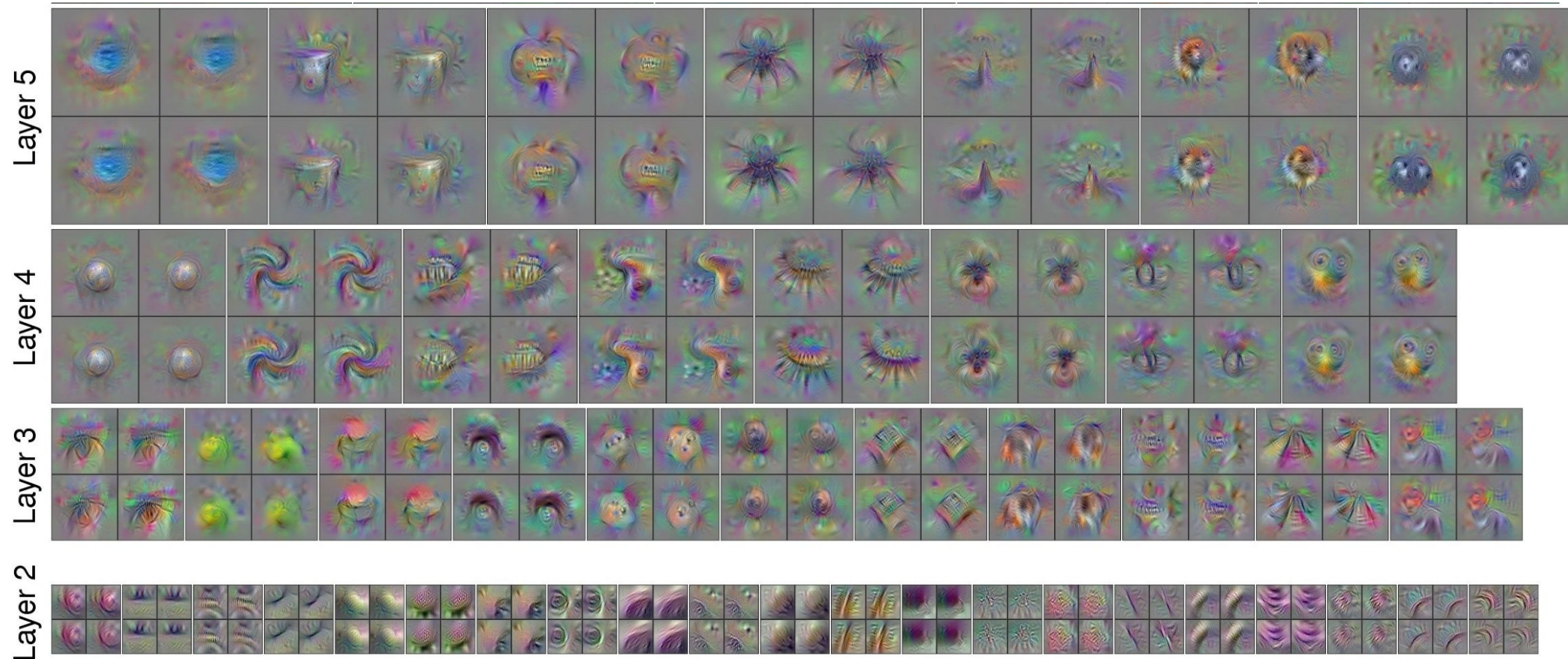
Station Wagon



Black Swan

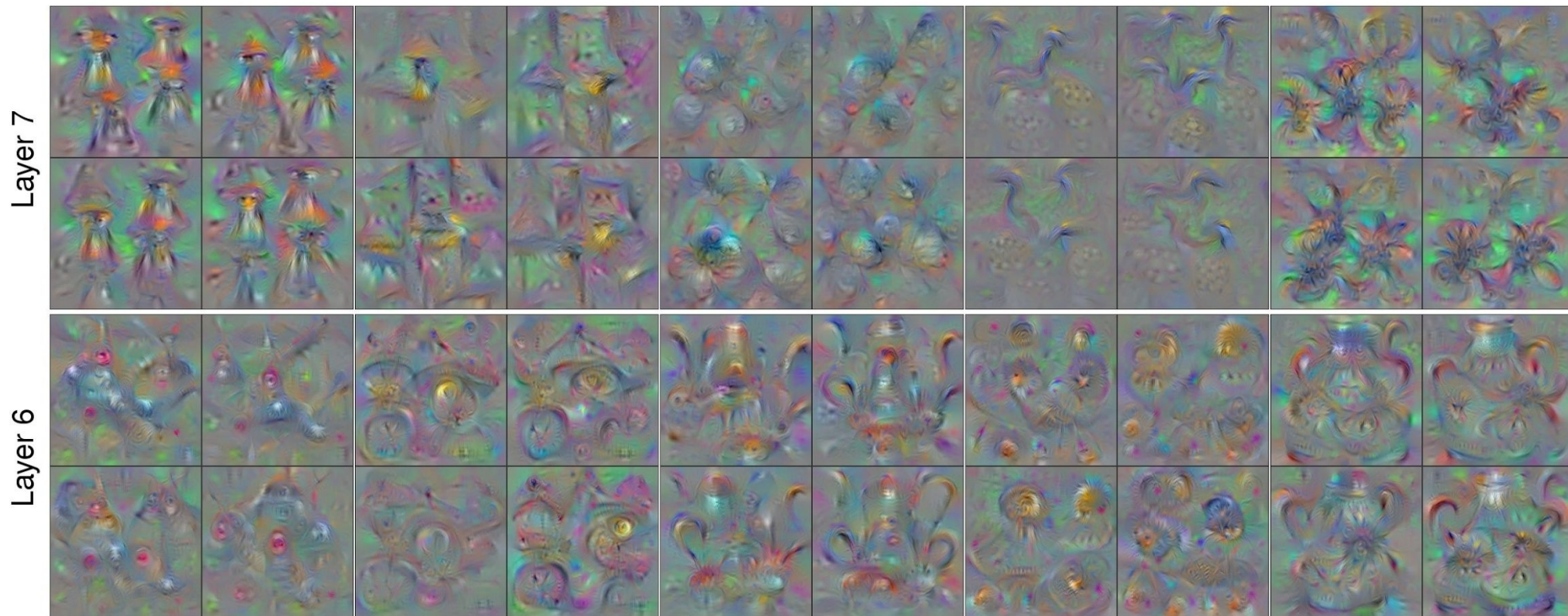
Visualizing CNN features: Gradient Ascent

Use the same approach to visualize intermediate features



Visualizing CNN features: Gradient Ascent

Use the same approach to visualize intermediate features



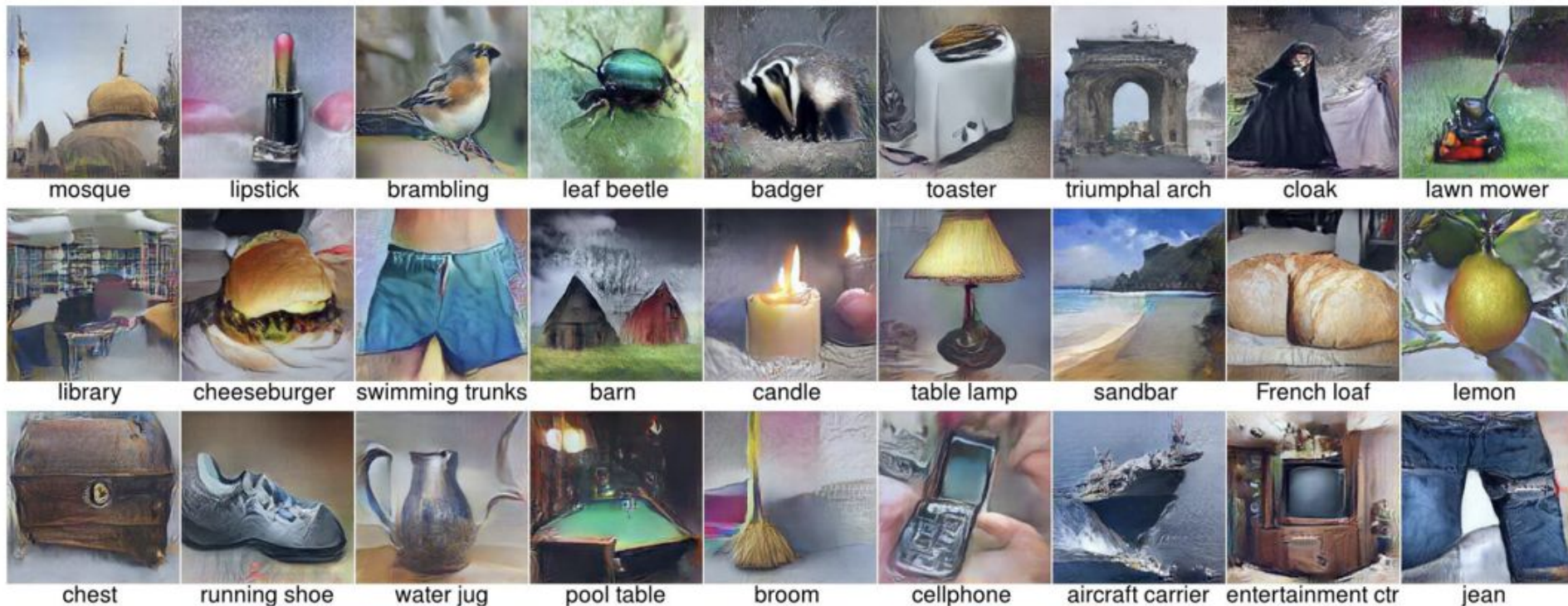
Visualizing CNN features: Gradient Ascent

You can add even more tricks to get nicer results:



Visualizing CNN features: Gradient Ascent

GAN image priors give amazing results:



Feature Inversion

Given a feature vector for an image, find a new image such that:

- Its features are similar to the given features
- It “looks natural” (image prior regularization)

Feature Inversion

Given a feature vector for an image, find a new image such that:

- Its features are similar to the given features
- It “looks natural” (image prior regularization)

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^{H \times W \times C}} \ell(\Phi(\mathbf{x}), \Phi_0) + \lambda \mathcal{R}(\mathbf{x})$$

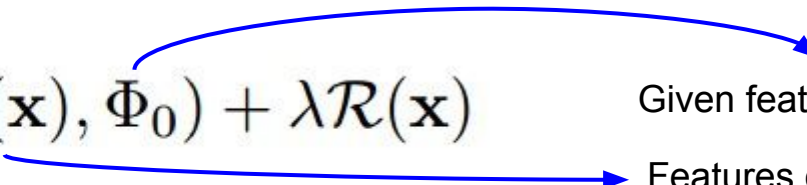
$$\ell(\Phi(\mathbf{x}), \Phi_0) = \|\Phi(\mathbf{x}) - \Phi_0\|^2$$

$$\mathcal{R}_{V^\beta}(\mathbf{x}) = \sum_{i,j} \left((x_{i,j+1} - x_{ij})^2 + (x_{i+1,j} - x_{ij})^2 \right)^{\frac{\beta}{2}}$$

Feature Inversion

Given a feature vector for an image, find a new image such that:

- Its features are similar to the given features
- It “looks natural” (image prior regularization)

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^{H \times W \times C}} \ell(\Phi(\mathbf{x}), \Phi_0) + \lambda \mathcal{R}(\mathbf{x})$$


Given feature vector

Features of new image

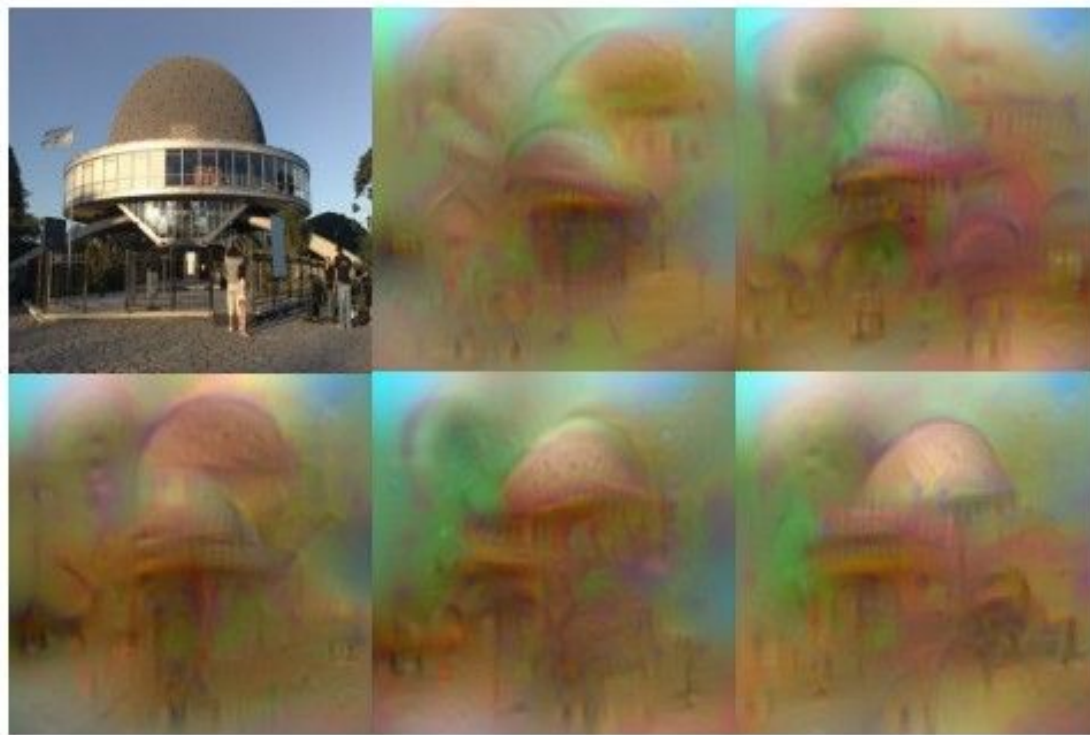
$$\ell(\Phi(\mathbf{x}), \Phi_0) = \|\Phi(\mathbf{x}) - \Phi_0\|^2$$

$$\mathcal{R}_{V^\beta}(\mathbf{x}) = \sum_{i,j} \left((x_{i,j+1} - x_{ij})^2 + (x_{i+1,j} - x_{ij})^2 \right)^{\frac{\beta}{2}}$$


Total Variation regularizer
(encourages spatial smoothness)

Feature Inversion

original image



Reconstructions
from the 1000
log probabilities
for ImageNet
(ILSVRC)
classes

Feature Inversion

Reconstructions from the representation after last last pooling layer
(immediately before the first Fully Connected layer)

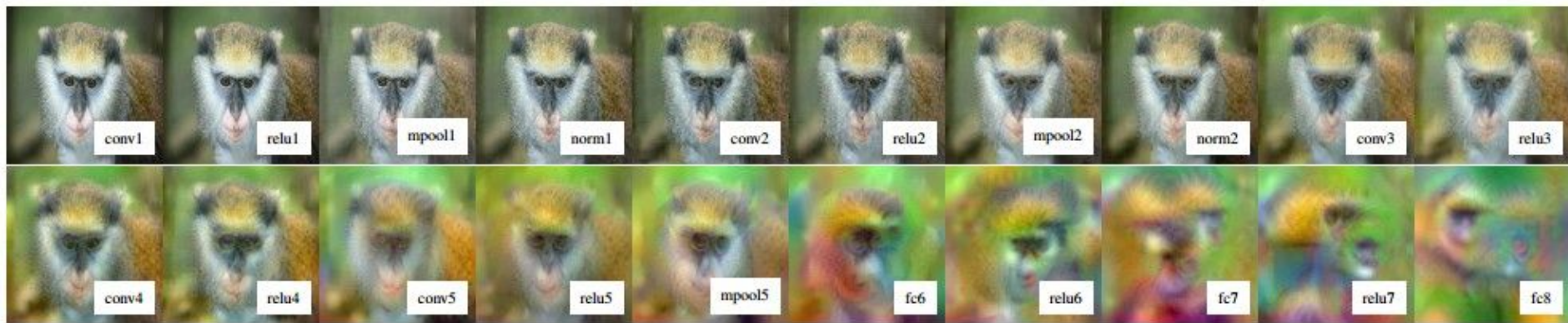


Feature Inversion



Reconstructions from intermediate layers

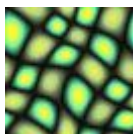
Higher layers are less sensitive to changes in color, texture, and shape



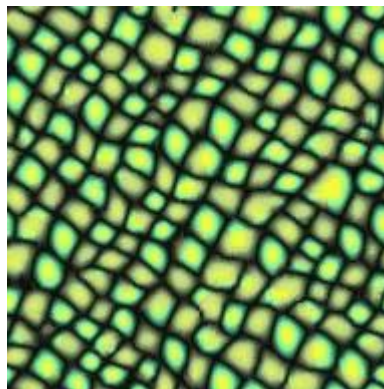
(Neural) Texture Synthesis

Texture Synthesis

Given a sample patch of some texture, can we generate a bigger image of the same texture?



Input



Output